Phase 1: Data Processing and Normalization

I processed the MDDS dataset which contained state-wise Excel files. All files were merged into a single dataset with over 5,64,000 rows.

Data cleaning was performed by removing unnecessary columns and duplicates.

The dataset was then structured into normalized tables:

State, District, Sub-District, and Village.

These tables maintain hierarchical relationships and are stored as CSV files for further use.

The dataset follows a hierarchical structure:

Country → State → District → Sub-District → Village,

enabling efficient data organization and future scalability.

Project Title: Village Search Backend API

Overview:

This project is a scalable backend system built using Node.js, Express, Prisma, and PostgreSQL (Neon). It models hierarchical Indian administrative data.

Modules:

- State

- District

- SubDistrict

- Village

Features:

- Bulk import of large dataset (~564,000 villages)

- Optimized database schema with relationships

- Pagination for large data handling

- Search API with autocomplete functionality

- Full address hierarchy output

APIs:

- /api/v1/states

- /api/v1/districts

- /api/v1/subdistricts

- /api/v1/villages?page=1&limit=20

- /api/v1/search?q=keyword

Code Structure:

- Prisma schema defines relational models

- Express routes handle API logic

- Import script handles large dataset ingestion

- Search route performs optimized filtering

Future Improvements:

- Add frontend UI
- Add filters (state, district)
- Implement caching for faster responses